

Lecture 3

OPTIMIZERS 1: INVARIANCES (AND HYPERPARAMETERS)

CS312

Tatsu H

Logistics

Assignments and Quiz

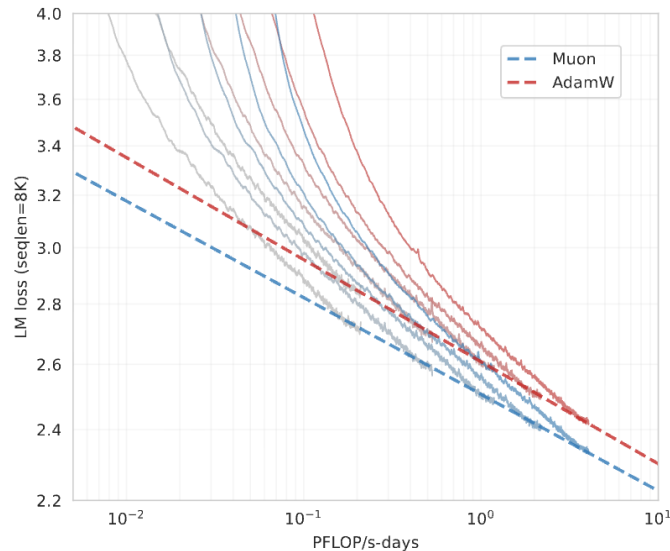
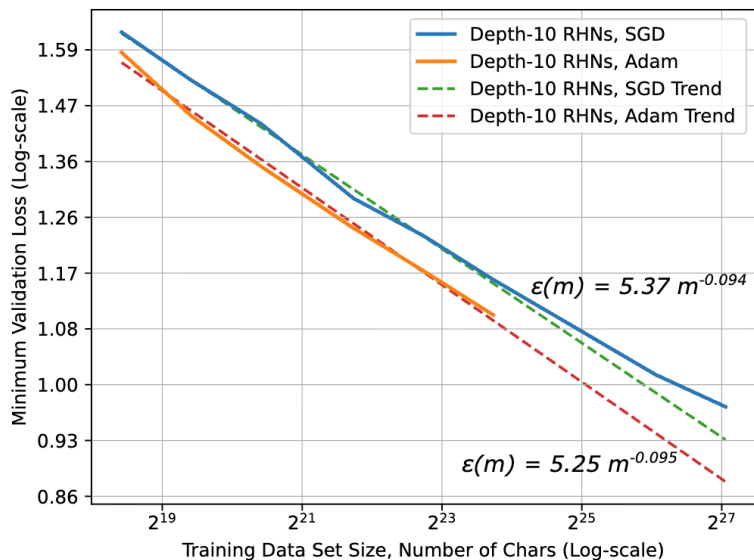
- A2 due Tuesday next week (~1.5 weeks)
- Q2 at same time

Office hours and tutorials

- Tutorial scheduling next week.
- If you know you'll drop, please do it before then!

Optimization is central to training

Training is about making the loss go down, and optimizers are what make that happen



A general purpose, robust improvements in optimizers would be *incredibly* valuable

A classic view of first order optimization

Optimization under a ‘first order’ oracle (access to function value + gradient)

- (i) A collection $\mathcal{F}$ of convex functions $f : \mathbb{R}^n \rightarrow \mathbb{R}$
- (ii) A closed convex set $C \subset \mathbb{R}^n$ over which we optimize
- (iii) A *stochastic gradient oracle*, which consists of a sample space $\mathcal{S}$, a gradient mapping

$$g : \mathbb{R}^n \times \mathcal{S} \times \mathcal{F} \rightarrow \mathbb{R}^n,$$

and (implicitly) a probability distributions P on $\mathcal{S}$. The stochastic gradient oracle may be *queried* at a point x , and when queried, draws $S \sim P$ with the property that

$$(5.1.1) \quad \mathbb{E}[g(x, S, f)] \in \partial f(x).$$

Depending on the scenario of the problem, the optimization procedure may be given access either to S or simply the value of the stochastic gradient $g = g(x, S, f)$, and the goal is to use the sequence of observations $g(x_k, S_k, f)$, for $k = 1, 2, \dots$, to optimize f .

Many beautiful methods (mirror descent, frank wolfe, etc) that we will ignore in favor of
Ugly ol minibatch stochastic gradient descent

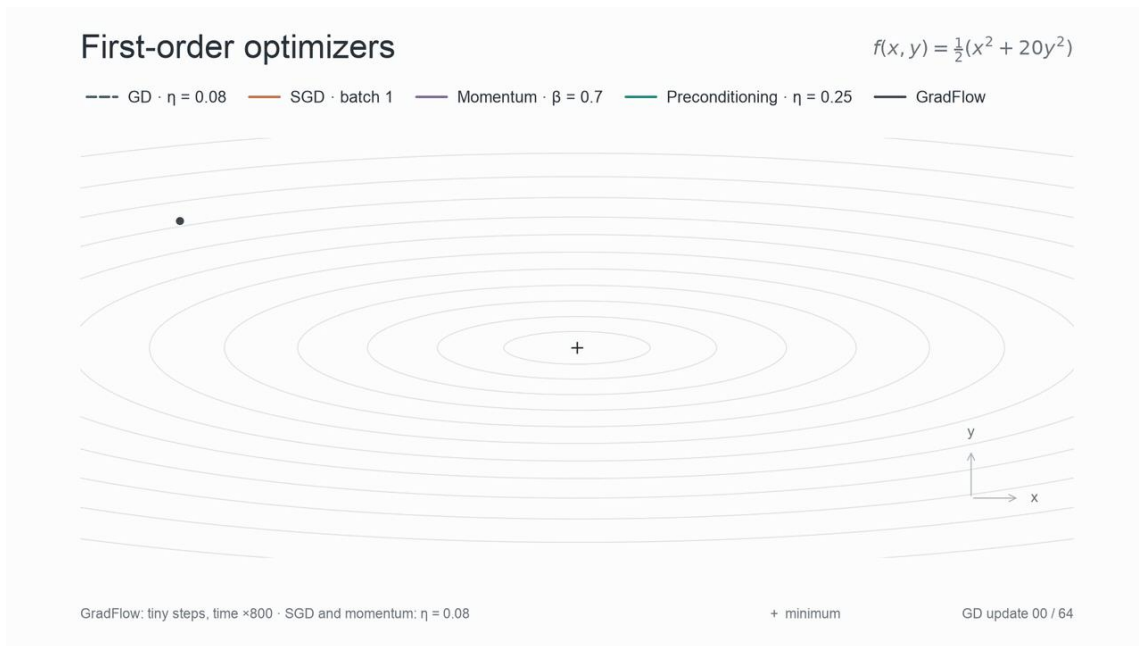
Different parts of an optimizer

The basic “SGD update” is extremely simple

$$\theta_t = \theta_{t-1} - \eta \nabla \ell_t(\theta_{t-1})$$

But, important variations!

- Learning rate
- Batching
- Preconditioners
- Momentum



Learning rate and Batching

First choice:

Learning rates

- Do we want to be in the 'smooth' regime?

Batch size

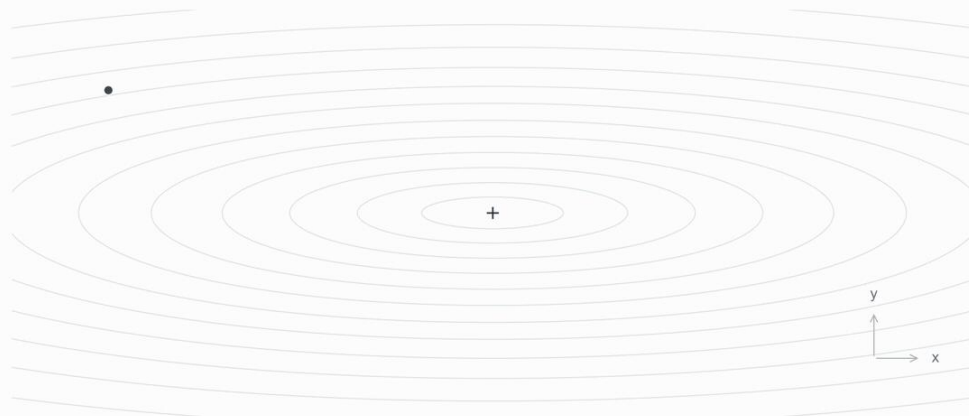
- What's the right tradeoff for stochasticity

(And how does it relate with scale?)

Learning rate & minibatching

$$f(x, y) = \frac{1}{2}(x^2 + 20y^2)$$

— Full batch · $\eta = 0.006$ - - - Full batch · $\eta = 0.08$ — Full batch · $\eta = 0.104$
— Batch 1 · $\eta = 0.08$ — Batch 32 · $\eta = 0.08$ — GradFlow



GradFlow: $\eta = 0.0001 \cdot 800$ tiny steps per GD update

+ minimum

GD update 00 / 64

Preconditioning

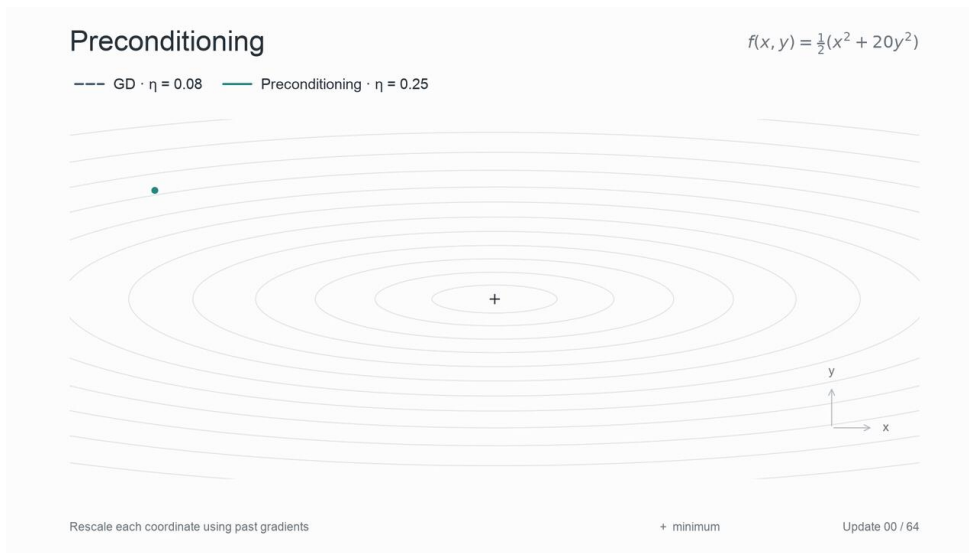
Preconditioned GD: ‘adjust’ the gradients

$$\theta_t = \theta_{t-1} - \eta \mathbf{A}_t \nabla \ell_t(\theta_{t-1})$$

The preconditioner is like a ‘change of coordinate’

- Enables faster or slower LR per param
- Allows us to ‘decorrelate’ parameters
- **Can be learned ‘on the go’*** (basis of most modern optimizers)

Huge impact in practice



* Preconditioners must be PSD with bounded min/max eigenvalues

Momentum

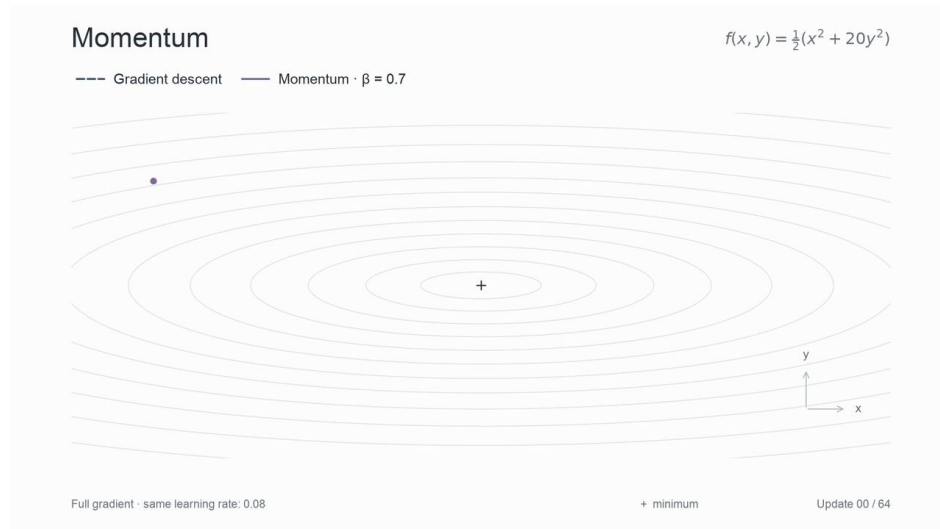
Momentum GD: accumulate gradients

$$\theta_t = \theta_{t-1} - \eta \nabla \ell_t(\theta_{t-1}) + \mu(\theta_{t-1} - \theta_{t-2})$$

Momentum allows models to adapt to aligned gradients

- Same gradients over many steps accelerate steps
- Striking improvements in convergence rates (NAG)*

Also huge impact in practice



*From $1/T$ to $1/T^2$ rate on smooth functions using Nesterov

A few (important) questions that we study

Progress in optimization is often careful treatment of these ideas.

We will cover a few useful (and tractable) questions in this space

How does batch interact with preconditioner/momentum? (Today, NQM)

How should we define or think about the LR? (Today, norms).

Per layer learning rates (and inits)? (next week, muP)

What preconditioner? (next week, Muon and friends)

From classic first order to deep learning optimizers

Basic SGD

$$\theta_t = \theta_{t-1} - \eta \nabla \ell_t(\theta_{t-1})$$

AdamW

$$m_{t+1} = \beta_1 m_t + (1 - \beta_1) g_t, \quad v_{t+1} = \beta_2 v_t + (1 - \beta_2) g_t^2$$

$$\widehat{m}_{t+1} = \frac{m_{t+1}}{1 - \beta_1^{t+1}}, \quad \widehat{v}_{t+1} = \frac{v_{t+1}}{1 - \beta_2^{t+1}}$$

$$U_t = \frac{\widehat{m}_{t+1}}{\widehat{v}_{t+1} + \epsilon}$$

$$W_{t+1} = W_t - \eta_t U_t - \eta_t \lambda W_t$$

SGD with some carefully chosen **momentum** and **preconditioning**

The 'W' in AdamW refers to the fact that the decay is in the update, not the loss

Why are 'deep learning' optimizers different?

1. Loose solutions fast (not great solutions slow)

Optimization theory traditionally cared about *first order stationarity* and *small ϵ*
Deep learning optimizers care about neither..

2. Doesn't need to work for everything

We *only* care about fast optimization of deep neural networks. We can narrow the scope to very specific subsets of the set of objective functions

Does adam converge?

Well known that original proof was wrong (with explicit, non-convergent examples)

Published as a conference paper at ICLR 2018

ON THE CONVERGENCE OF ADAM AND BEYOND

Sashank J. Reddi, Satyen Kale & Sanjiv Kumar

Google New York

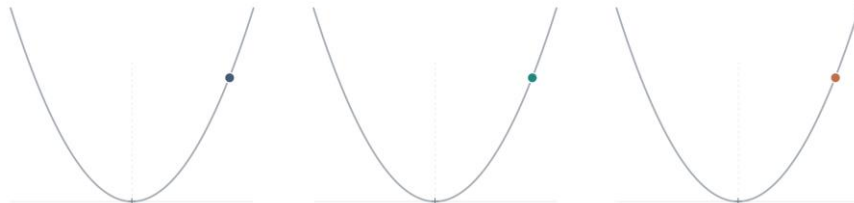
New York, NY 10011, USA

{sashank, satyengkale, sanjivk}@google.com

SGD

Nesterov

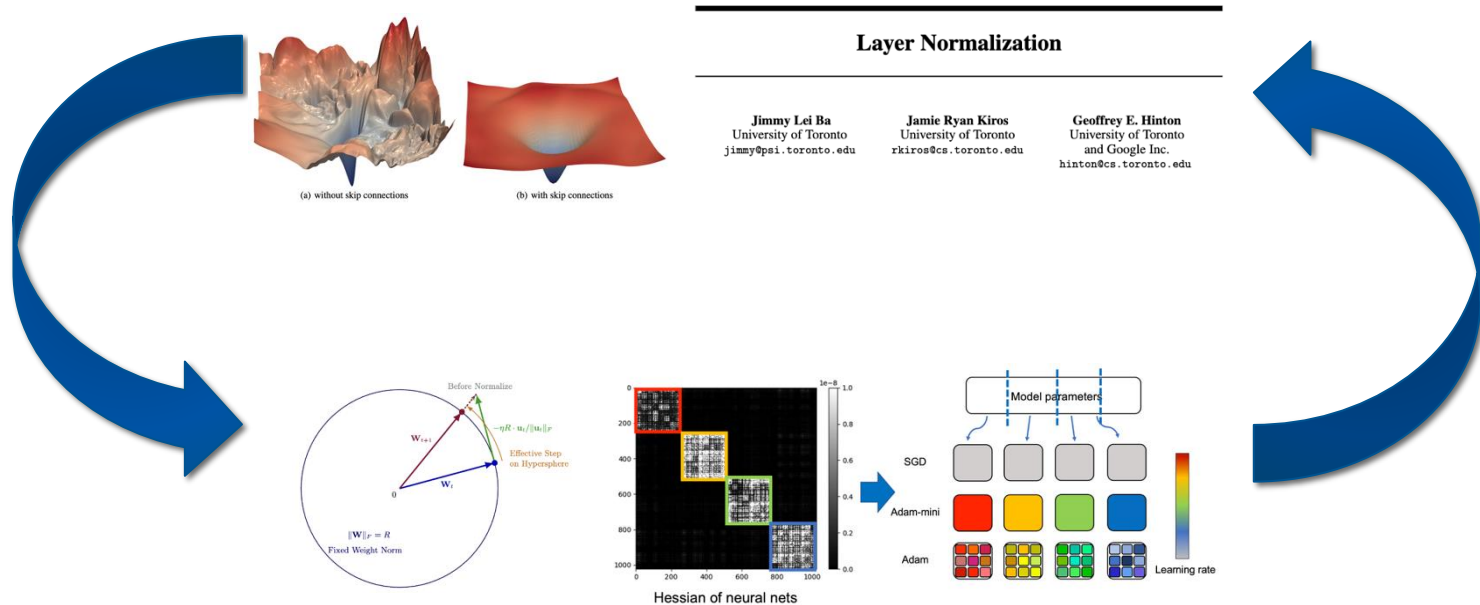
Adam



But we also don't care! (we never converge our LLMs anyway)

Implications for optimizer architecture codesign

Design architecture find easy to optimize losses



Design optimizer to accelerate training

Optimizers are complicated..

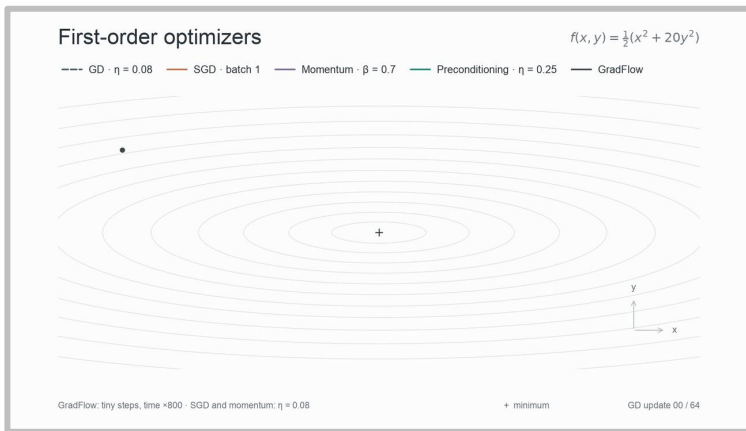
Rough mindset guide

- Optimizers are not about ‘classic convex optimization math’
 - Things that should not work (Adam) do work well!
 - But also intuition from convex optimization can be useful
- Get some ‘top down’ understanding of macro scale behaviors
 - How do optimizers react to LR?
 - How do preconditioners and Hessians interact?
- Work backwards to reconcile ‘top down’ and ‘bottom up’ views
 - How can we control one-step feature learning?

Lecture today

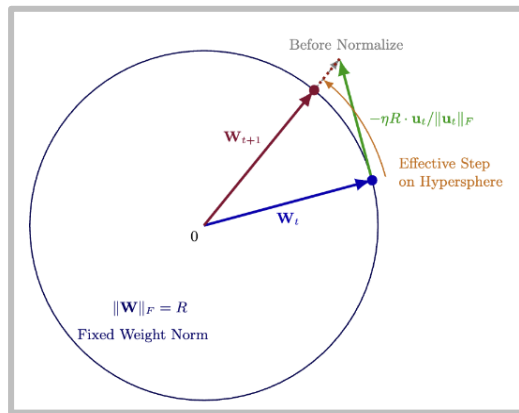
Part 1

Convex intuitions (and their limits)



Part 2

Optimizers and parameter norms



Part 1: The noisy quadratic model

How far can we push the ‘world is a quadratic’ view?

Advantages of the quadratic view

- Many important quantities (i.e. optimal batch) are exactly solvable
- In convex world, quadratics are very useful to think about (i.e. preconditioning)

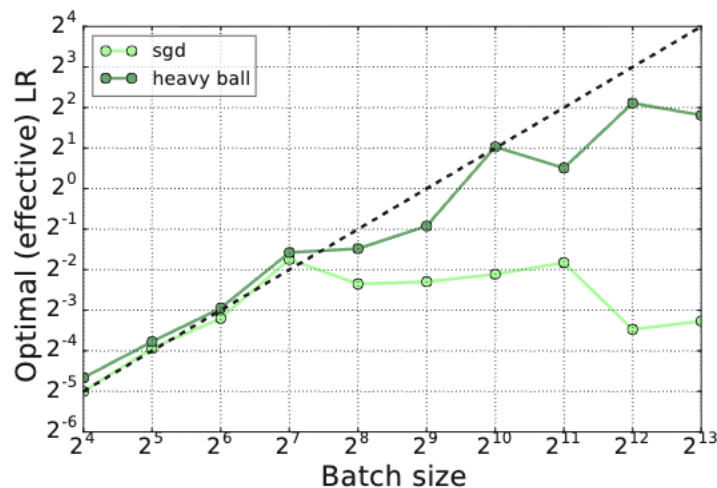
We will go through a fairly influential paper that takes this approach

Which Algorithmic Choices Matter at Which Batch Sizes? Insights From a Noisy Quadratic Model

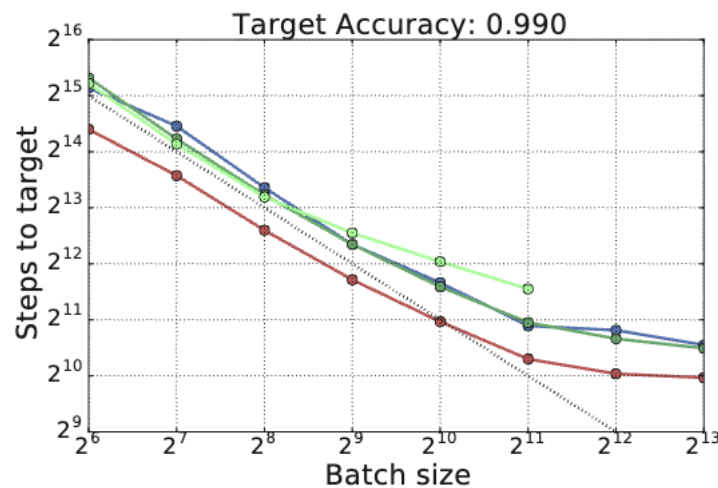
Guodong Zhang^{1,2,3*}, Lala Li³, Zachary Nado³, James Martens⁴,
Sushant Sachdeva¹, George E. Dahl³, Christopher J. Shallue³, Roger Grosse^{1,2}
¹University of Toronto, ²Vector Institute, ³Google Research, Brain Team, ⁴DeepMind

Why NQM?

Surprisingly accurate predictions



Batch vs LR on CIFAR



Number of steps needed
(i.e. 'critical batch')

(dashed lines indicate theory predictions. theory predicts deviation from the line at large batch)

Defining the NQM

The objective function

$$L(\theta) = \frac{1}{2} \theta^\top H \theta = \sum_i h_i \theta_i^2$$

(the second eq. is via diagonalization and change of variables. think of h_i as eigenvalues)

The gradients

$$g(\theta) = H\theta + \epsilon$$

$$\epsilon \sim N(0, C)$$

(C is assumed to be simultaneously diagonalizable with H)

Batching

$$\epsilon_B \sim N(0, C/B)$$

(Batching reduces the noise in our gradients via i.i.d averaging)

Dynamics under the NQM

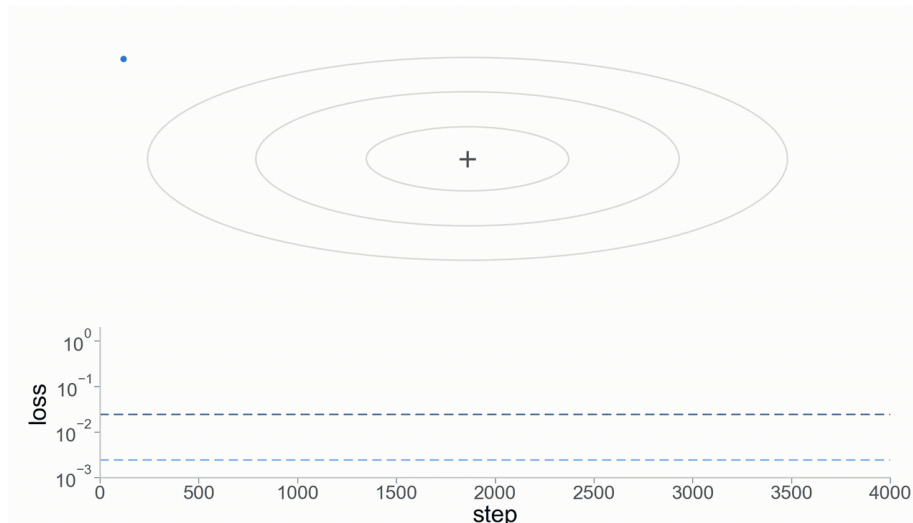
The NQM assumptions mean coordinates are separable

The dynamics are 'simple' and evolve as

$$\theta_i(t+1) = (1 - \alpha h_i) \theta_i(t) + \alpha \sqrt{c_i/B} \epsilon_i$$

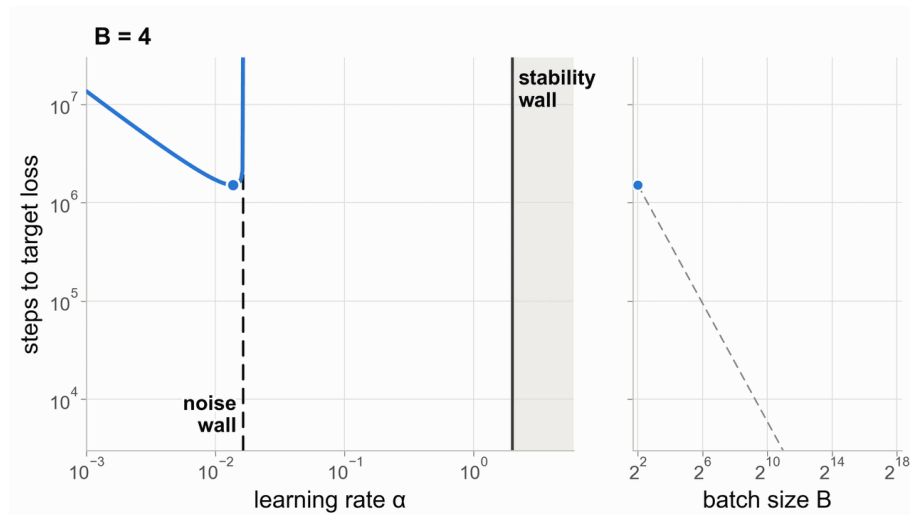
Expected risks converge **rapidly** to a **steady state**

$$E[\ell(\theta_i(t))] = (1 - \alpha h_i)^{2t} E[\ell(\theta_i(0))] + (1 - (1 - \alpha h_i)^{2t}) \frac{\alpha c_i}{2B(2 - \alpha h_i)}$$



Batch size vs learning rate dynamics

NQM predicts that up to a certain point, learning rate should go up *linearly* with batch



But learning rate cannot go up forever! For a quadratic, $2/h$ is the max stable LR
After this point, batch shows rapidly diminishing returns (the ‘critical batch’)

Implications for momentum

What happens to momentum SGD?

Now consider the momentum update,

$$\begin{aligned}m_i(t+1) &= \beta m_i(t) + h_i \theta_i(t) + \sqrt{c_i/B} \epsilon_i, \\ \theta_i(t+1) &= \theta_i(t) - \alpha m_i(t+1).\end{aligned}$$

We can derive the expected risk

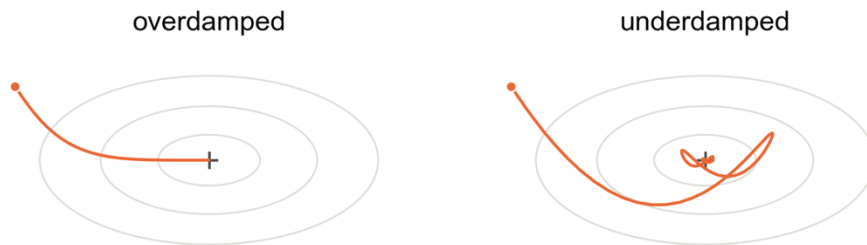
Theorem 1. *Given a dimension index i , and $0 \leq \beta < 1$ with $\beta \neq (1 - \sqrt{\alpha h_i})^2$, the expected risk at time t associated with that dimension satisfies the upper bound*

$$\mathbb{E}[\ell(\theta_i(t))] \leq \left(\frac{(r_1^{t+1} - r_2^{t+1}) - \beta(r_1^t - r_2^t)}{r_1 - r_2} \right)^2 \mathbb{E}[\ell(\theta_i(0))] + \frac{(1 + \beta)\alpha c_i}{2B(2\beta + 2 - \alpha h_i)(1 - \beta)}, \quad (6)$$

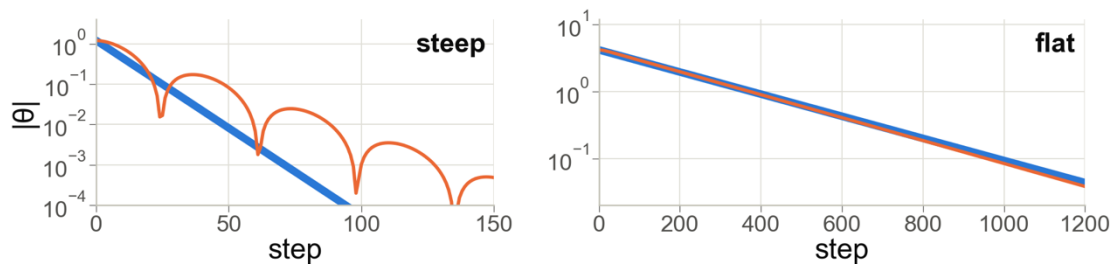
where r_1 and r_2 (with $r_1 \geq r_2$) are the two roots of the quadratic equation $x^2 - (1 - \alpha h_i + \beta)x + \beta = 0$.

Understanding momentum dynamics in NQM

Overdamped vs Underdamped

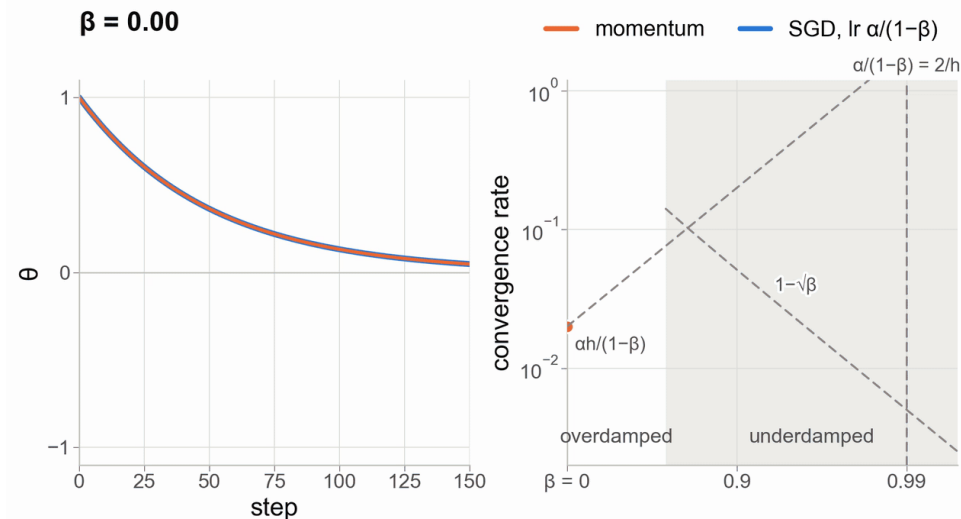


Rate limiting by high curvature directions



Damping, momentum, and convergence rates

How does over vs underdamped affect convergence relative to plain SGD?

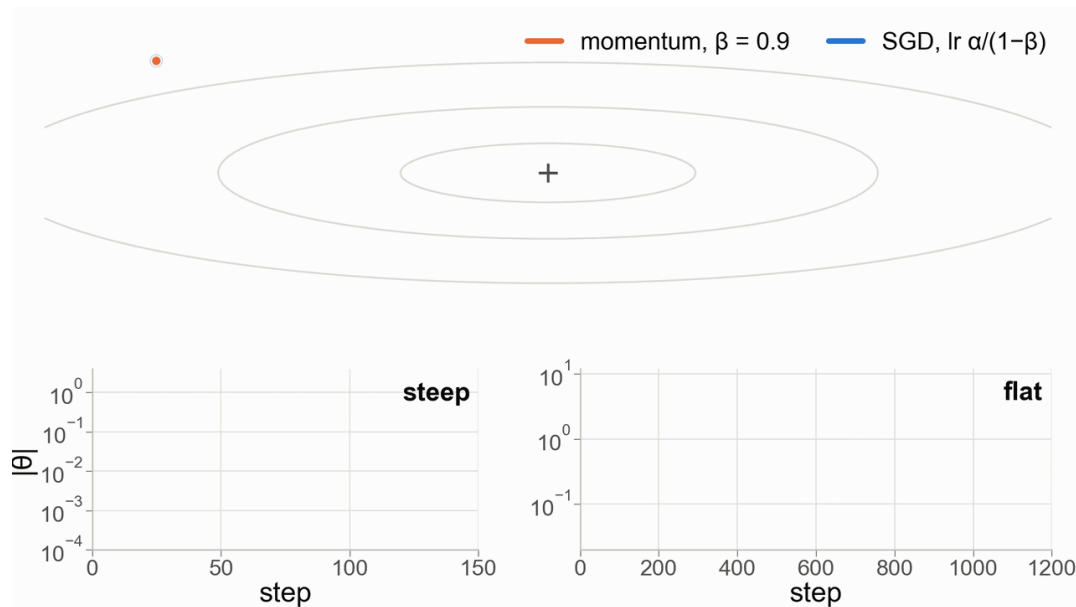


- Momentum has *fastest* convergence at the damping transition point
- In the overdamped case, there exists a LR that matches momentum's convergence rate
- There is an upper limit to SGD LR due to the sharpness limit

(momentum acceleration allows optimizers to have higher effective LR than this limit)

Overdamped dynamics control the convergence

The optimal momentum always has one overdamped coordinate
If SGD w/ effective step is convergent, this matches momentum long-run



Implications for momentum and batch

Theorem 1. *Given a dimension index i , and $0 \leq \beta < 1$ with $\beta \neq (1 - \sqrt{\alpha h_i})^2$, the expected risk at time t associated with that dimension satisfies the upper bound*

$$\mathbb{E}[\ell(\theta_i(t))] \leq \left(\frac{(r_1^{t+1} - r_2^{t+1}) - \beta(r_1^t - r_2^t)}{r_1 - r_2} \right)^2 \mathbb{E}[\ell(\theta_i(0))] + \frac{(1 + \beta)\alpha c_i}{2B(2\beta + 2 - \alpha h_i)(1 - \beta)}, \quad (6)$$

where r_1 and r_2 (with $r_1 \geq r_2$) are the two roots of the quadratic equation $x^2 - (1 - \alpha h_i + \beta)x + \beta = 0$.

1. In the small batch case:

Irreducible error is large, and step size must be small to compensate.

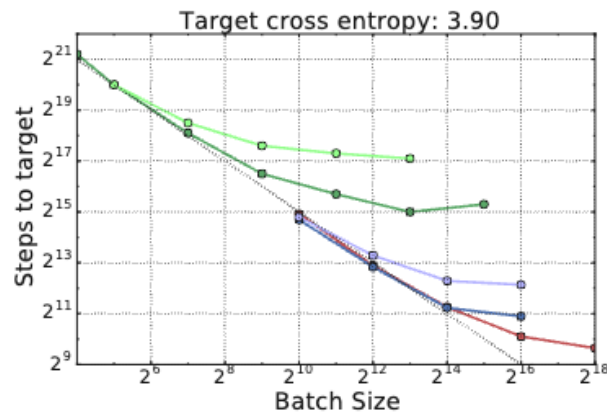
Larger effective step size for momentum is *counterproductive*

2. In the large batch case:

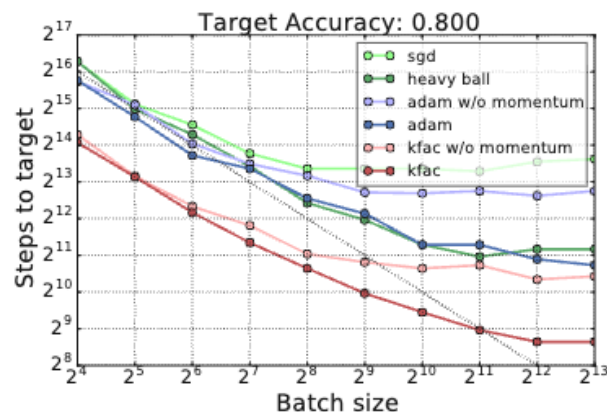
Low irreducible error, large step size

Sufficiently large momentum reaches unattainable effective step sizes

Empirical support for the large-batch-momentum view



(f) Transformer on LM1B



(c) ResNet8 on CIFAR10

At low batch sizes: no benefit to momentum

At large batch sizes: significant gains from momentum (heavy ball) as well as other tricks

Other papers on momentum and batch size

Small Batch Size Training for Language Models: When Vanilla SGD Works, and Why Gradient Accumulation Is Wasteful

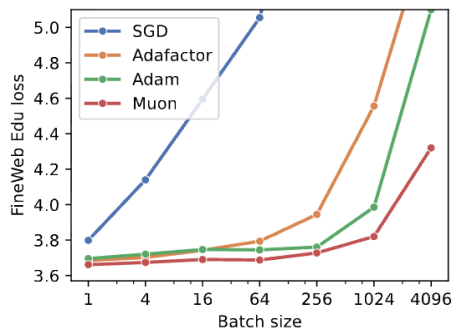
Martin Marek
New York University
martin.m@nyu.edu

Sanae Lotfi
New York University

Aditya Somasundaram
Columbia University

Andrew Gordon Wilson
New York University

Micah Goldblum
Columbia University



(a) Large batches require more sophisticated optimizers

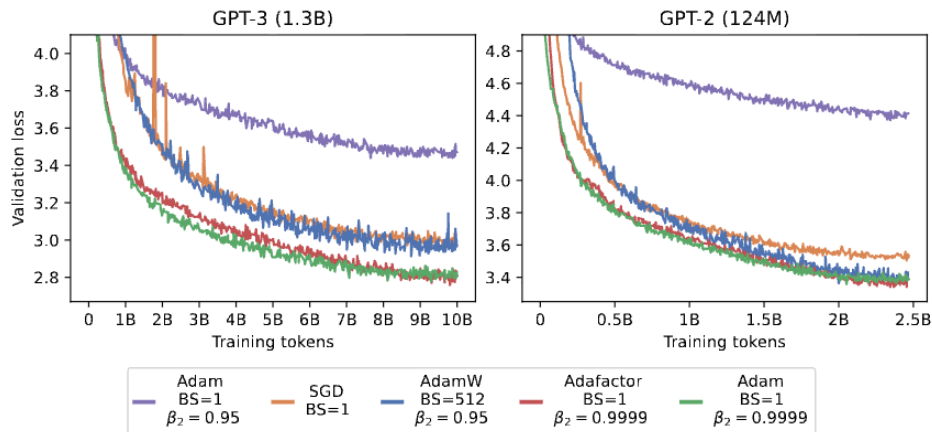


Figure 6: **Vanilla SGD performs competitively at larger model scales with minimal tuning.** We train GPT-3 (1.3B) and GPT-2 (124M) on FineWeb and compare validation loss across optimizers. **Left:** For GPT-3 (1.3B), vanilla **SGD** with batch size 1 and no momentum performs on par with the **AdamW** configuration from Brown et al. [1], despite having no optimizer state. Our proposed β_2 scaling rule improves Adam’s performance at small batch sizes and outperforms the baseline. **Right:** After tuning the learning rate of each optimizer on GPT-2 (124M), **Adam** and **Adafactor** at batch size 1 match the performance of **AdamW** at batch size 512, while **SGD** performs slightly worse.

NQM and preconditioning

Recall that preconditioned gradient descent for the NQM follows

$$\boldsymbol{\theta}(t + 1) = \boldsymbol{\theta}(t) - \alpha \mathbf{P}^{-1} [\mathbf{H}\boldsymbol{\theta} + \boldsymbol{\epsilon}]$$

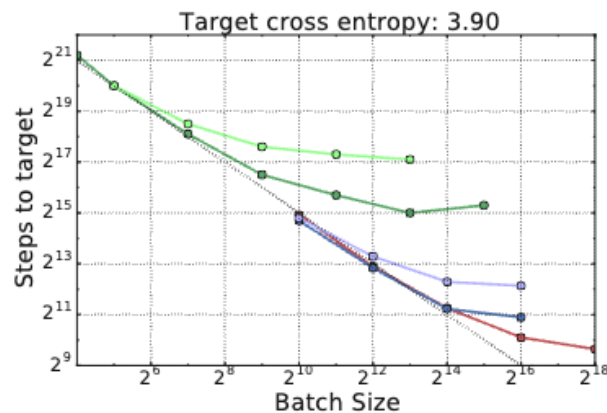
The NQM authors study a very simple model of weak preconditioning, $P = H^p$

This is nice since it immediately gives a simple form of the risk

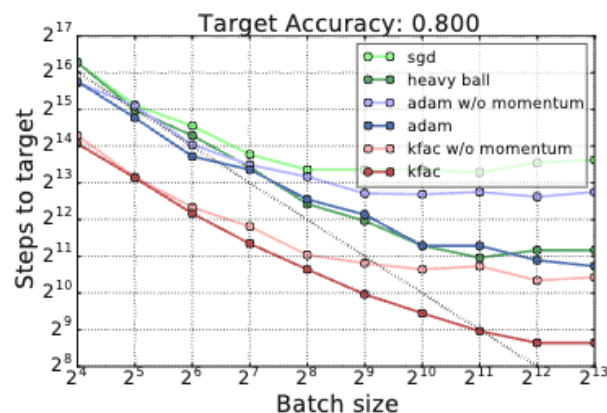
$$\mathbb{E} [\mathcal{L}(\boldsymbol{\theta}(t))] \leq \sum_{i=1}^d (1 - \alpha h_i^{(1-p)})^{2t} \mathbb{E} [\ell(\theta_i(0))] + \sum_{i=1}^d \frac{\alpha c_i h_i^{-p}}{2B(2 - \alpha h_i^{1-p})}.$$

Notice how the first term decreases with higher p (up to 1.0), the second term increases.

Comparisons on a range of optimizers



(f) Transformer on LM1B



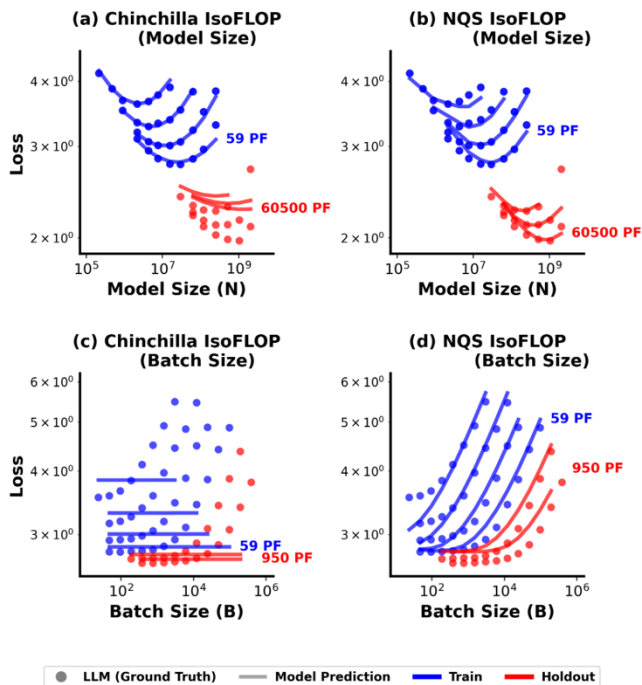
(c) ResNet8 on CIFAR10

At low batch: no benefits to Adam vs others

At large batch sizes: significant gains from Adam (and KFAC)

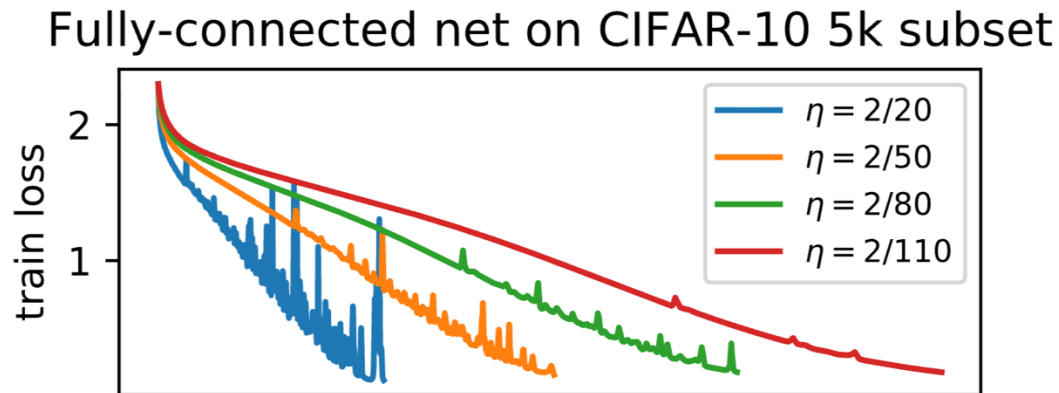
Other NQM stuff.

Recent extensions on using NQM style methods for scaling laws (li et al)



Limits of convex, quadratics

But... don't trust quadratics blindly! *Lots* of deep learning is not like quadratics



Why do high learning rate, batch gradient descent trajectories show noise-like patterns?

We will cover many of these subtleties later..

Recap: NQM

The noisy quadratic model is very useful for thinking about optimization

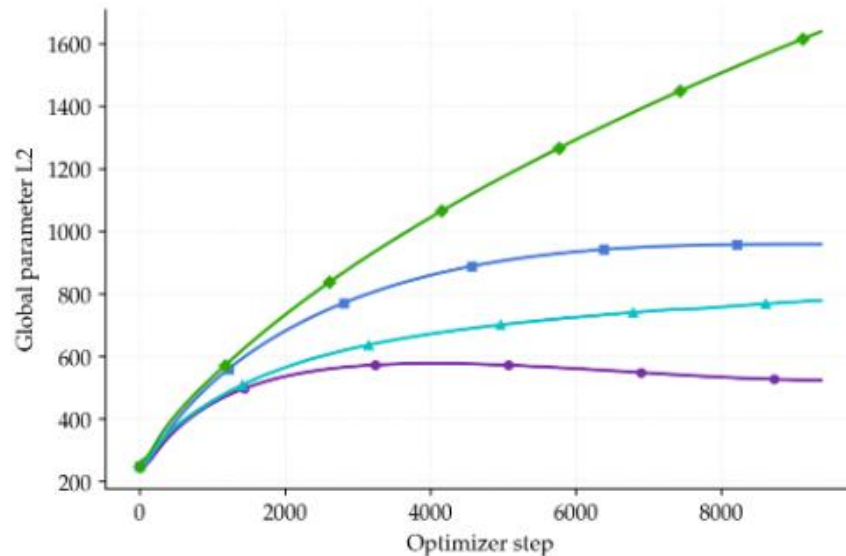
Very convenient model for solving various properties analytically

Good rough intuition for when momentum or preconditioning helps

But, we know the world isn't quadratic. Knowing the 'right' model and limits are key

Part 2: Norms, weight decay, hyperball

In quiz 1, we've already seen some fun norm effects..



We know that norms (in our transformer model organism) tend to grow over time.

1. Why?
2. Who cares?

Preliminaries: Norm invariance in transformers

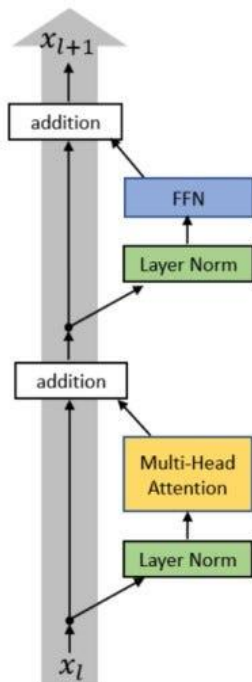
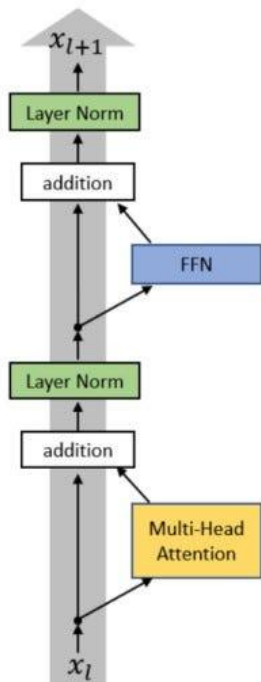


Figure from Xiong 2020

Transformers have many *norm invariant* parts

Recall the rmsnorm operation

$$\text{Norm}(x) = \frac{x}{\sqrt{\|x\|_2^2 + \epsilon}} \gamma$$

This operation (nearly) kills the norm of x
If we have

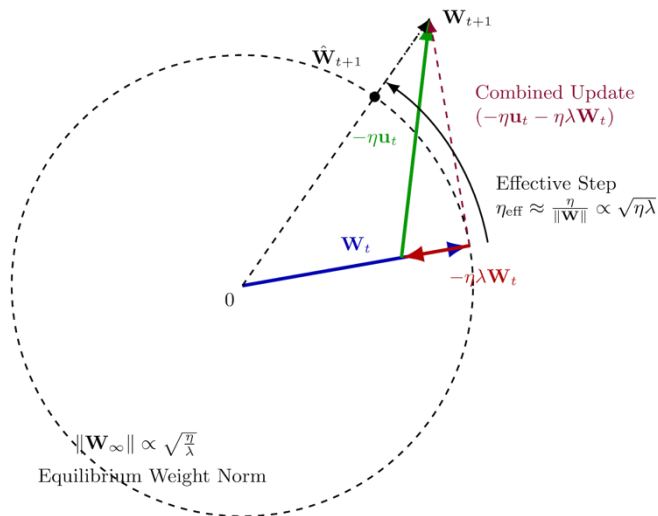
$$\text{Loss}(\text{Norm}(Ax))$$

The loss is *scale invariant* to the scale of A

Norm drift in norm invariant models

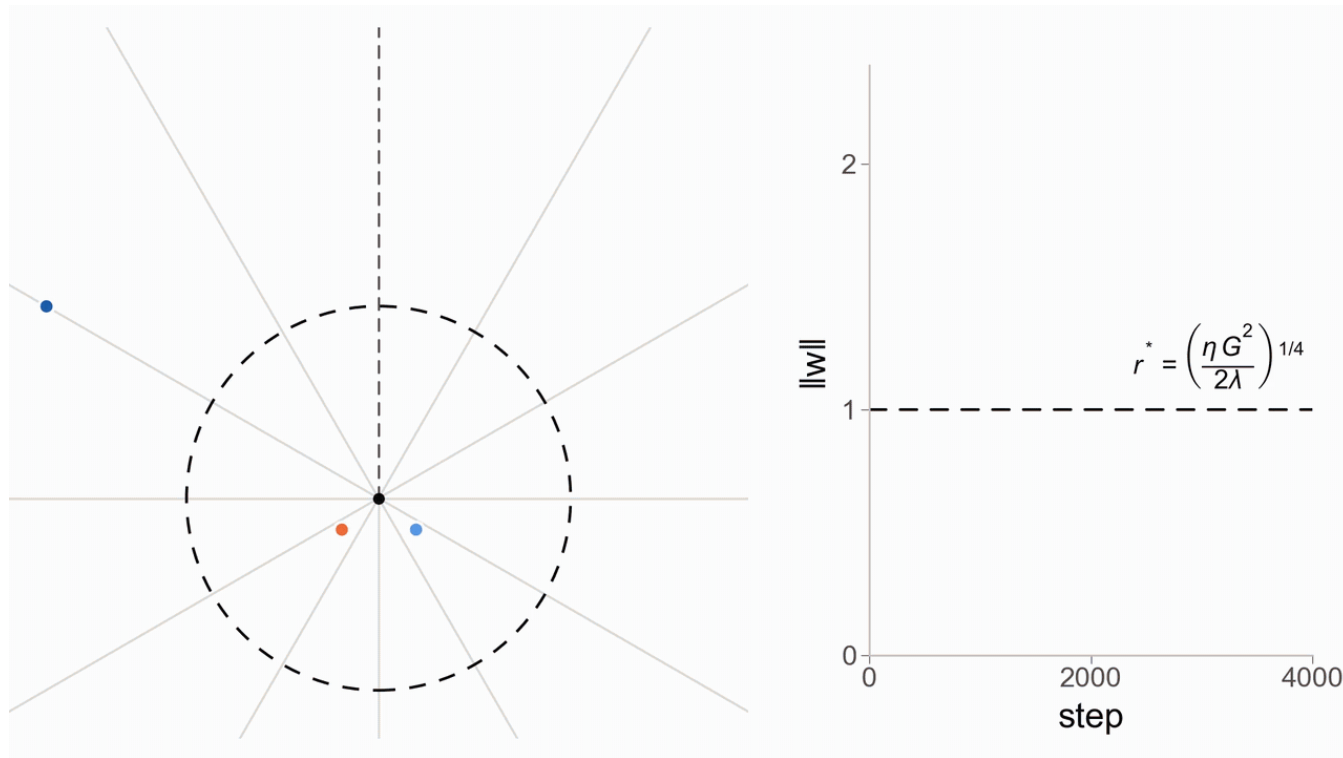
For norm invariant weights, we might expect the following..

- Gradients are tangent to the current norm (or approx. for approx. invariant)
- This increases the norm, with higher LR increasing the norm more
- Weight decay pulls it back, with a dynamic equilibrium



Norm drift in the toy case

We can see this in a simple, 2D norm invariant loss

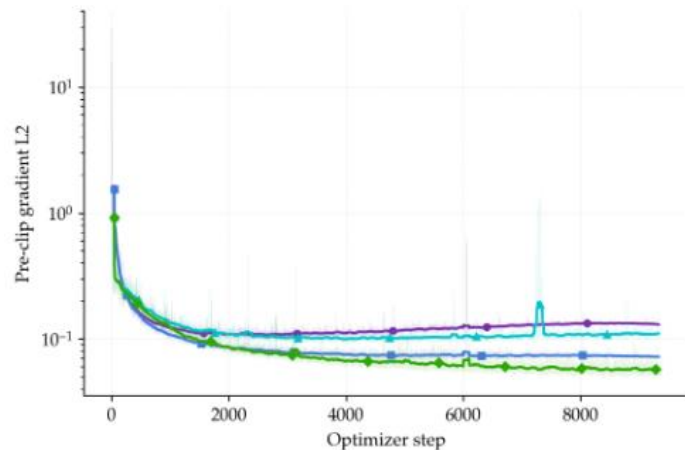
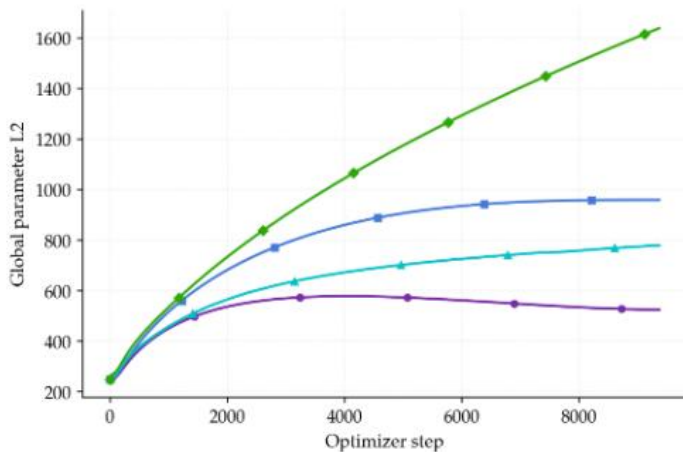


Do we care if the norm drifts?

Who cares if the norm drifts?

Larger norm leads to smaller gradients

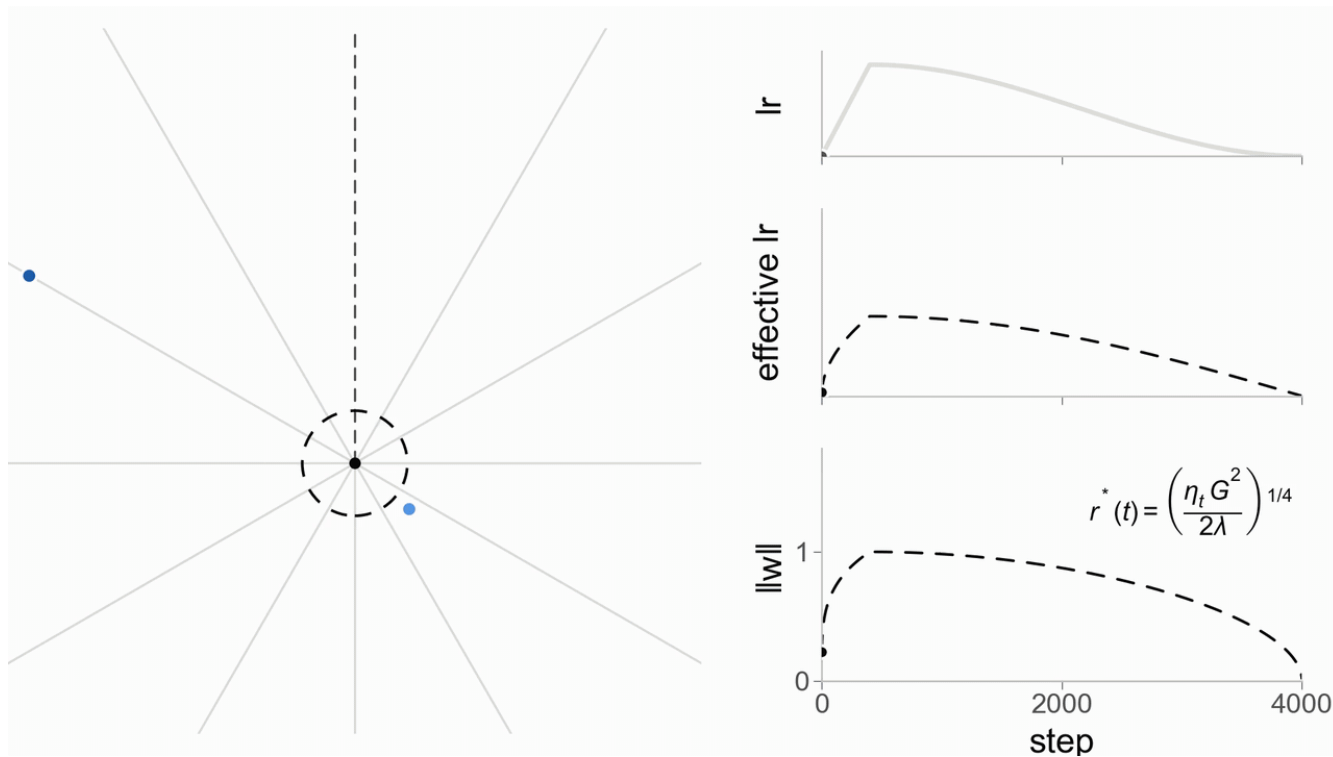
Effective step size $\frac{\eta}{\|\theta\|_2^2}$ decays, and is uncontrolled



$$\hat{\theta}_l^{t+1} \leftarrow \hat{\theta}_l^t - \eta \|\theta_l^t\|_2^{-2} (\mathbf{I} - \hat{\theta}_l^t \hat{\theta}_l^{t\top}) \nabla_{\theta_l} \mathcal{L}(\hat{\theta}^t) + O(\eta^2). \quad (9)$$

Back to our toy case

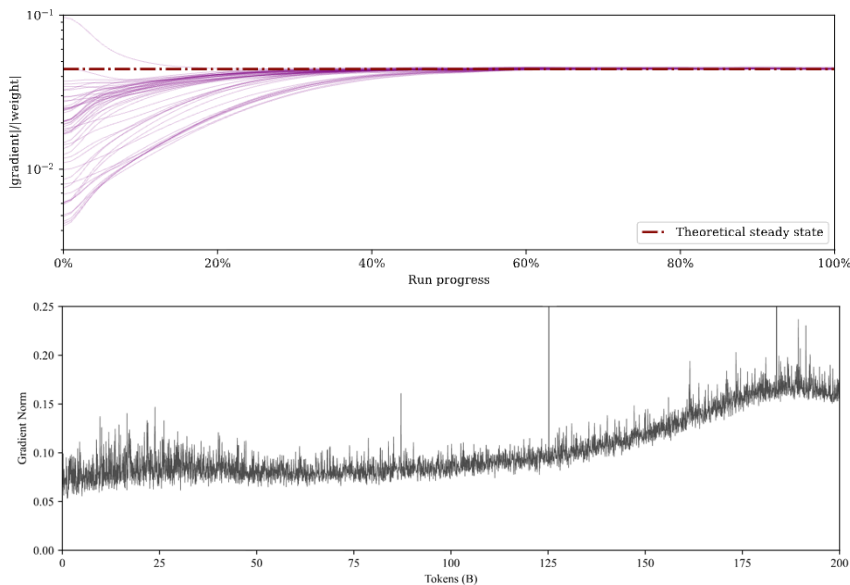
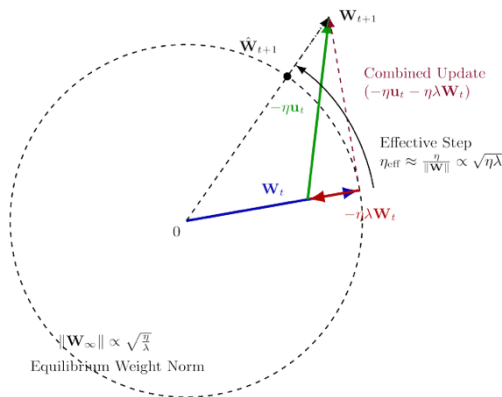
How does the ‘effective’ learning rate change in our toy example?



Revisiting weight decay

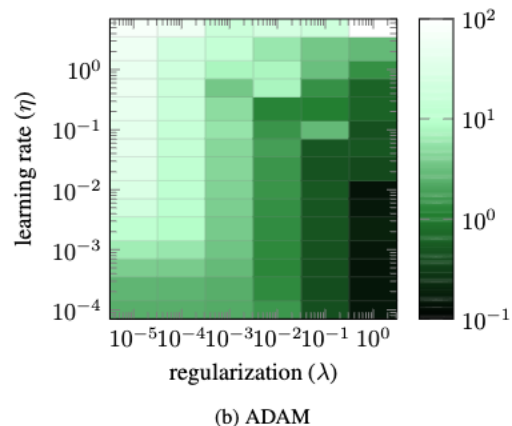
Other views: Defazio 2025

- Ratio of gradient to weight should equilibrate to $\Theta\left(\sqrt{\frac{2\lambda}{\alpha}}\right)$.
- Vary weight decay over time as $\lambda_t = \frac{\lambda \alpha_t}{\alpha_{max}}$ to keep the ratio constant



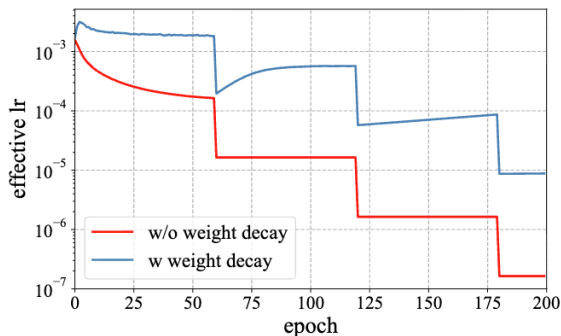
Evidence for this view

LR and WD jointly control norm



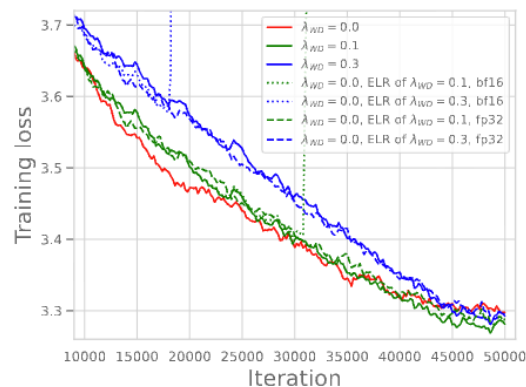
van Laarhoven 2017

WD manipulates effective LR



Guodong et al 2019

WD improves loss in one pass SGD



D'Angelo et al 2024

N.b. Weight decay has other impacts, like regularizing the jacobian norm

Designing a new optimizer

What might we do to deal with this annoying weight decay and LR coupling?

Implicit Learning Rate Decay

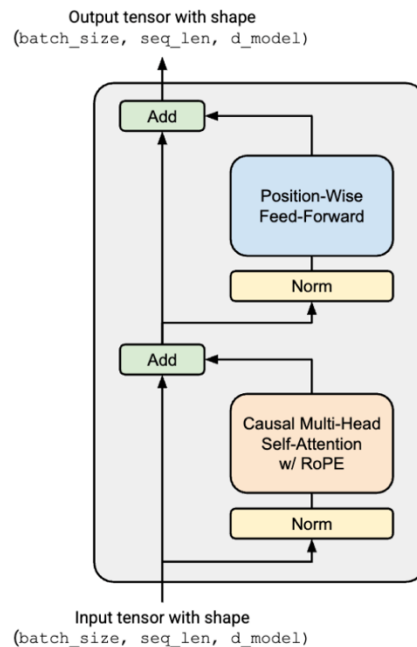
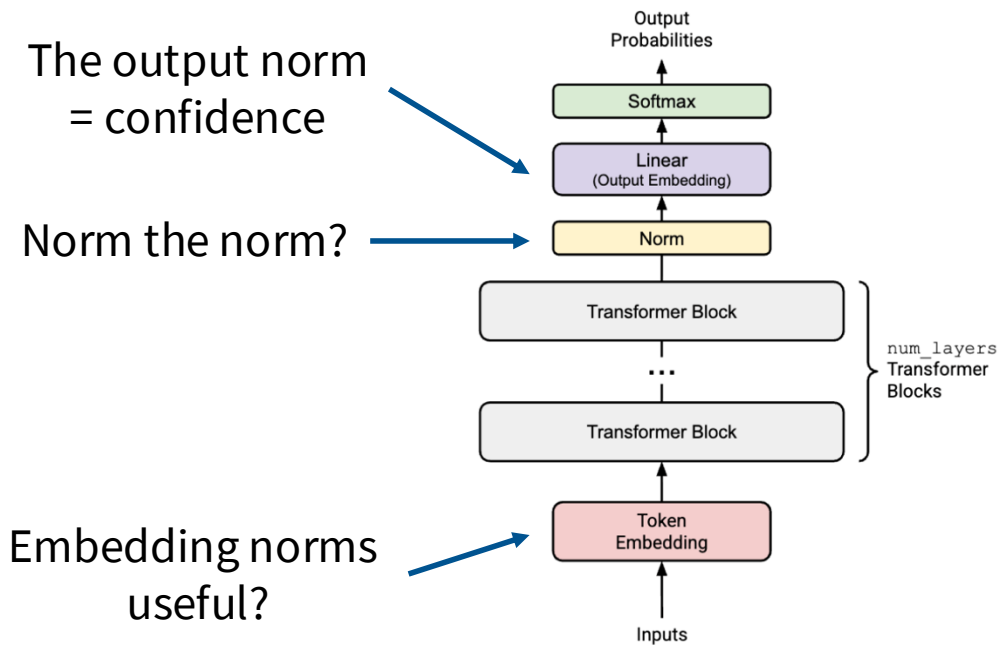
Counteracting the implicit LR effect:

- Exponentially increasing LR schedule (Li and Arora, 2020)
- Use weight decay (up next!)
- Explicitly normalize each $\mathbf{w}_j$ to unit norm
 - Then the effective LR is just α
 - This is not common practice, but I expect it could eliminate a lot of experimental confounds

Sounds a bit silly... but is it?

Subtleties

We don't literally want a norm-constrained model..



Normalize the 'internal' weights (MLP and attention weights)

Hyperball

Kaiyue Wen[†] Xingyu Dang[‡] Kaifeng Lyu[§] Tengyu Ma[†] Percy Liang[†]
kaiyuew@stanford.edu xingyu.dang@princeton.edu klyu@mail.tsinghua.edu.cn
tengyuma@stanford.edu pliang@cs.stanford.edu

Algorithm 1 Hyperball wrapper for a parameter matrix W

- 1: **Input:** parameter matrix W_t , base optimizer $\mathcal{O}$, optimizer state $\mathcal{S}_t$, radius R , schedule $\{\eta_t\}$
 - 2: Compute base optimizer update $u_t, \mathcal{S}_{t+1} \leftarrow \mathcal{O}(\nabla_{W_t} L(W_t), \mathcal{S}_t)$
 - 3: Set normalized update direction $\hat{u}_t \leftarrow \text{Normalize}(u_t)$
 - 4: Take unprojected step $\widetilde{W}_{t+1} \leftarrow W_t - \eta_t R \hat{u}_t$
 - 5: Project to radius R : $W_{t+1} \leftarrow R \cdot \text{Normalize}(\widetilde{W}_{t+1})$
-

We can ‘wrap’ any optimizer with a norm-invariance aware wrapper

- Project our updates to be unit norm (line 3)
- Update, scaling up to norm R and using step size η
- Project the resulting parameter back to norm R

As a reasonable default, set $R = \|W_0\|_F$

Many successful related ideas,

Reparametrization

Weight Normalization: A Simple Reparameterization to Accelerate Training of Deep Neural Networks

Tim Salimans
OpenAI
tim@openai.com

Diederik P. Kingma
OpenAI
dpkingma@openai.com

Columnwise weight norm Activation norm

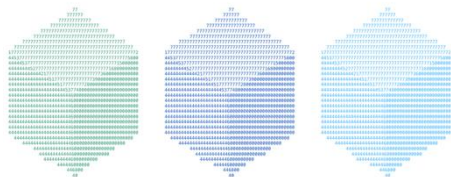
NGPT: NORMALIZED TRANSFORMER WITH REPRESENTATION LEARNING ON THE HYPERSPHERE

Ilya Loshchilov, Cheng-Ping Hsieh, Simeng Sun & Boris Ginsburg
NVIDIA
{iloshchilov, chsieh, simengs, bginsburg}@nvidia.com

Gradient descent on a Stiefel manifold (vs spherical)

Modular Manifolds

Jeremy Bernstein
Sep 26, 2025



Ball projection (like hyperball)

Controlled LLM Training on Spectral Sphere

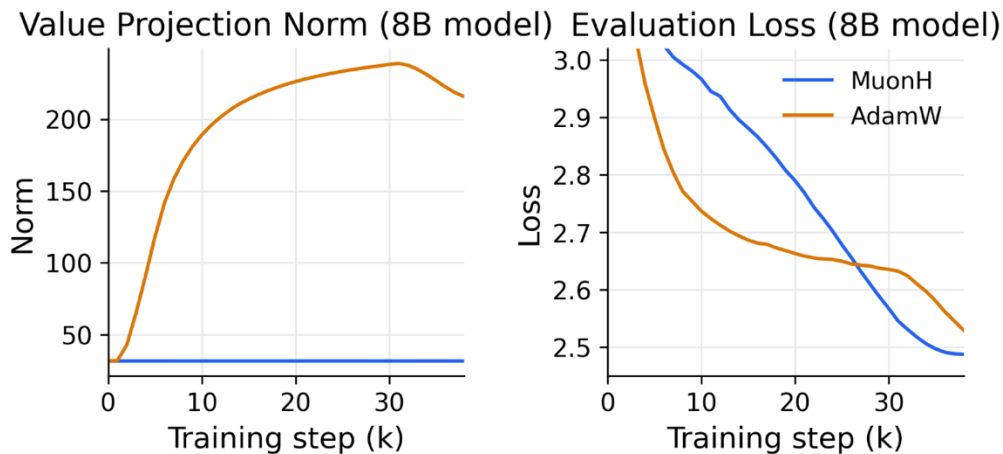
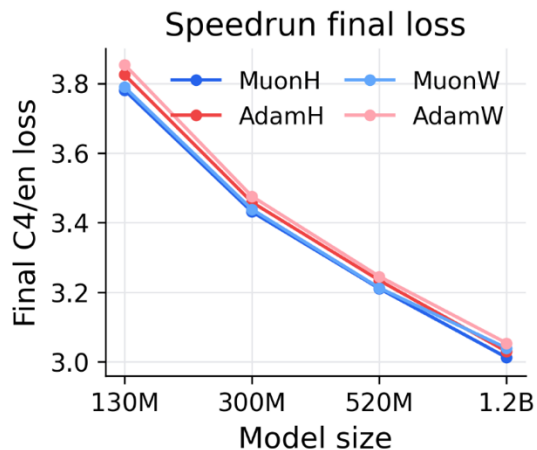
Tian Xie^{1*} Haoming Luo² Haoyu Tang² Yiwen Hu² Jason Klein Liu⁴ Qingnan Ren¹

Yang Wang¹ Wayne Xin Zhao^{2,4} Rui Yan³ Bing Su² Chong Luo¹ Baining Guo¹

¹Microsoft Research Asia ²Renmin University ³Wuhan University ⁴IQuest Research

 Project Page: [Spectral-Sphere-Optimizer](#)

How does it work?



Pretty good! Lower loss and norm control by construction [From Wen et al '26]

Hyperparameter transfer

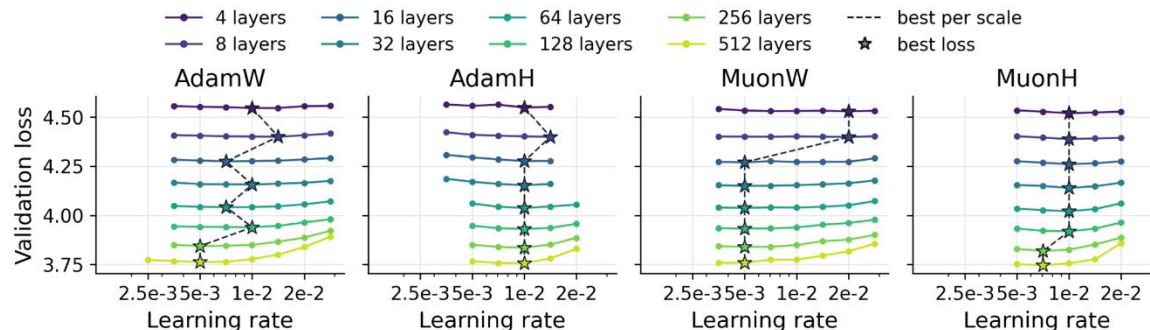


Figure 5: Depth scaling at fixed $d = 128$, 10B tokens per run. Each curve shows final validation loss versus learning rate for a given depth. Stars mark the best learning rate. Hyperball variants reduce the optimal learning rate drift across $L \in [4, 512]$ to $\approx 1.4\times$, versus $2\text{--}4\times$ for AdamW and MuonW.

Hyperball might also improve hyperparameter transfer
(You will try this in your assignment)

Alexander Hägele*
Machine Learning and Optimization Lab, EPFL

alexander.hagele@epfl.ch

Alejandro Hernández-Cano*
Machine Learning and Optimization Lab, EPFL

alejandro.hernandezcano@epfl.ch

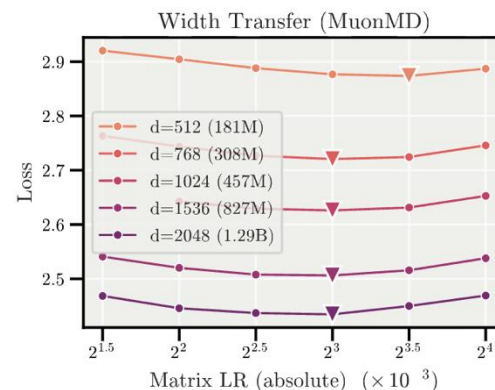
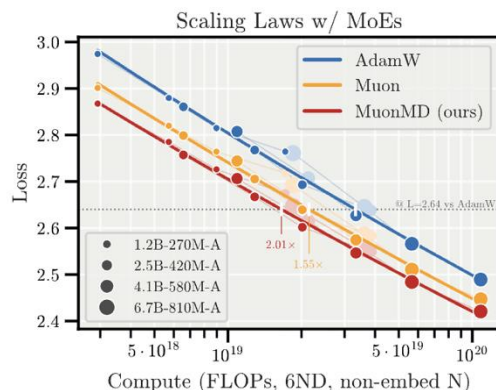
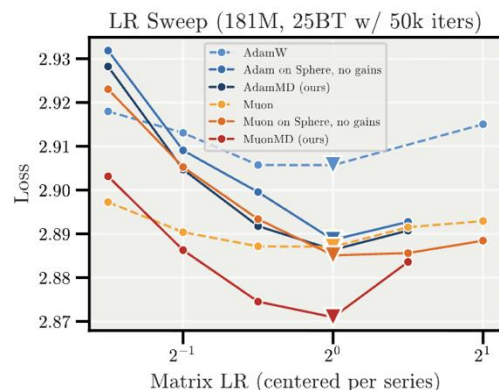
Atli Kosson†
Machine Learning and Optimization Lab, EPFL

atli.kosson@epfl.ch

Martin Jaggi†
Machine Learning and Optimization Lab, EPFL

martin.jaggi@epfl.ch

Some corroborating evidence



Experiments roughly line up with hyperball (plus some new stuff)

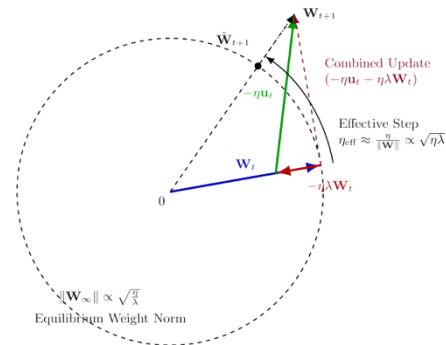
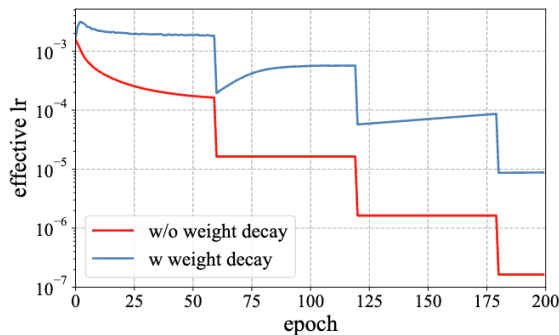
Also interesting evidence that frobenius norm (sphere projection) is the best norm

Optimizer architecture codesign

Momentum and preconditioner design

Optimization dynamics

Architecture and model constraints



Unit norm to control 'effective LR'

Recap: Norms

The quiz question we gave you has surprising implications!

In one-pass regimes, weight decay controls ‘effective LR’

Having the effective LR drift uncontrollably is often not ideal

Precise control over the norms (and therefore LR) can be very useful

Conclusion

Optimization (and deep learning) is complicated!

Theory intuitions can be useful! NQM is a workhorse for training dynamics

Mental models (i.e. quadratics) have limited scope. Knowing when they apply is subtle

From Roger Grosse's Toronto lectures on deep learning training dynamics..

- Much of what makes science challenging is determining what observations deserve to be studied as a single phenomenon.
 - The pipeline of “practitioners notice things, then theorists explain them” doesn't really work. You need a good mental model of what's going on even to know what experiments to run.